

ISTANBUL TECHNICAL UNIVERSITY
COMPUTER ENGINEERING DEPARTMENT

BLG 222E
COMPUTER ORGANIZATION
PROJECT 2 REPORT

CRN : 21335

LECTURER : Deniz Turgay Altılar

GROUP MEMBERS:

150210925 : NEJRA GUTIĆ

150220042 : ZEYNEP NUR YILMAZ - Group Representative

SPRING 2024

Contents

1	INTRODUCTION	1
1.1	Task Distribution	1
2	MATERIALS AND METHODS	1
2.1	Counter	1
2.2	Fetching and Decoding	1
2.3	Execution of the Operations	2
2.3.1	Operations with Address Reference	2
2.3.2	Operations without Address Reference	4
2.4	Resetting	6
3	RESULTS	6
4	DISCUSSION	8
5	CONCLUSION	8
	REFERENCES	9

1 INTRODUCTION

In this project, we were supposed to design a module (CPUSystem) that is able to read instructions from a memory file, perform them and write the output into either a register or a memory address when necessary. For performing the instructions, we were required to use the arithmetic logic unit system module that we have designed for the previous project. In our design, the whole process mainly takes place in the control unit module.

1.1 Task Distribution

For the majority of the project, it is hard to make a clear distinction between the parts that we worked on. But the only part we did an exact task distribution was the execution of the operations where the operations without address reference were implemented by Nejra Gutić and the ones with address reference were done by Zeynep Nur Yılmaz. The overall debugging was also done together.

2 MATERIALS AND METHODS

2.1 Counter

Firstly, to be able to control the performing times of the operations we created a counter module with the inputs clock, count, reset, and a 3-bit output. The counter counts up at every negative edge of the clock signal while count input value is 1 and the reset input value is 0. While reset is 1, the counter goes back to 0 and starts counting again.

But instead of using the counter as it is, we sent the output of the counter into a 3 to 8 decoder that we have designed (since the Tx in the CPUSystem simulation file is also 8 bits) and used its bits as the conditions of each operation (e.g. T[0], T[1], T[2]...). We created instances of both the counter and 3 to 8 decoder modules for this purpose in the control unit module where the main process takes place.

2.2 Fetching and Decoding

In the project description, due to the reason that the memory module outputs only an 8-bit value while the instruction register consists of 16-bits, it was required to fetch an instruction from the memory in 2 cycles. LSB 8-bit part of the instruction must be fetched during the first cycle and the MSB part in the second.

The fetching in our design takes place in the control unit module. At T[0], OutDSel of the address register file is set to PC to fetch the LSB of the instruction. Mem WR is set to 0 to read from memory and IR Write is set to 1 while IR LH is set to 0 to write to the LSB of the instruction register. PC is incremented by 1 at the end of this cycle. And at T[1], the same process is performed but with IR LH set to 1 to write to the MSB.

For decoding of the instruction, we designed an instruction decoder module that is activated with a T input. The instruction decoder takes IR as an input and outputs D (opcode), RSEL and address parts separately. SREG1, SREG2 and DSTREG were later taken from RSEL and address.

For the 36-bit D output, we used a 6 to 36 decoder inside of the instruction decoder to be able to use the bits of D similar to what we did with the counter. To explain further, D[0] is 1 for opcode 0x00, D[1] is 1 for 0x01, D[2] is 1 for 0x02 and so on. This way we were able to use these bits as conditions for our operations.

2.3 Execution of the Operations

The execution of all operations start at cycle T[3]. Since this part was done separately, it will be explained under 2 sections.

2.3.1 Operations with Address Reference

For these operations, the instruction was separated as the opcode D, register selector RSEL and the address.

OPCODE (6-bit)	RSEL (2-bit)	ADDRESS (8-bit)
----------------	--------------	-----------------

Figure 1: Format of addresses with Reference

For the operations that contain Rx, we used "case" for determining the value of either RF Regsel or RF OutASel according to the RSEL value.

For BRA, BNE and BNQ, the same operation $PC \leq PC + VALUE$ had to be performed with only a difference in the if conditions regarding the value of the Z flag (Z=0 for BNE, Z=1 for BNQ). To perform this operation, at T[3] we set MuxASel to 11 to send the lower bits of IR to the register file (RF) while also setting RF FunSel to 100 (only write low) and RF ScrSel to 0111 to write the value of IR into the scratch register S1. RF OutASel is set to 100 to send the value in S1 to the arithmetic logic unit (ALU). During this cycle, we also set ARF OutCSel to 00 to send PC to MUX A, set ALU FunSel to 10100 for addition and set MuxBSel to 00 to send ALUOut to the address register file (ARF). In the next clock cycle T[4], IR[7:0] is written into S1 so we set MuxASel to 01 to

send PC to RF, set RF FunSel to 010 (load) and RF ScrSel to 1011 to write PC into S2 while also setting RF OutBSel to 101 to send the value in S2 to the ALU. And finally at T[5], ARF FunSel is set to 010 (load) and ARF RegSel is set to 011 (PC) so the ALUOut is loaded into PC.

In POP operation, SP is incremented by 1 and M[SP] is written into Rx. To do this, at T[3] ARF RegSel is set to 110 (SP) and ARF FunSel is set to 001 (increment). Mem WR is set to 0 for reading from memory, MuxASel is set to 10 to send MemOut to RF and RF FunSel is set to 100 (only write low). At T[4], RF RegSel is set according to RSEL as explained in the beginning.

In PSH operation Rx is written into M[SP] and SP is decremented by 1. For doing so, at T[3] RF OutASel is set according to RSEL as explained before. ARF OutDSel is set to 11 (SP), Mem WR is set to 1 for writing into the memory, and MuxCSel is set to 0 (ALUOut[7:0]). ALU FunSel is set to 00000 (A) as default so the value from Rx is directly passed to memory. At T[4], ARF RegSel is set to 110 (SP) and ARF FunSel is set to 000 (decrement).

The operations MOVH and MOVL take one cycle to perform. At T[3], MuxASel is set to 11 to send IR[7:0] to RF, and RF RegSel is set according to RSEL as explained. For MOVH, RF FunSel is set to 110 to write into the higher bits of Rx and for MOVL it is set to 101 to write into the lower bits.

LDR operation also lasts only one cycle. At T[3], RF RegSel is set according to RSEL and RF FunSel is set to 100 (write low). ARF OutDSel is set to 10 (AR), Mem WR is set to 0 (read) and MuxASel is set to 10 (MemOut) so M[AR] is sent to RF and is written into Rx.

Execution of STR also takes one cycle. At T[3], RF OutASel is determined according to RSEL. ARF OutDSel is set to 10 (AR), Mem WR is set to 1 (write) and MuxCSel is set to 0. Since ALU FunSel is 00000 (A) by default, the lower bits of Rx are written into M[AR].

For BX, at T[3] ARF OutDSel is set to 11 (SP), ARF OutCSel is set to 00 (PC), MuxASel is set to 01 (OutC) and MuxCSel is set to 0. RF FunSel is set to 010 (load), RF ScrSel is set to 0111 (S1) and RF OutASel is set to 100 (S1) so the PC value can be loaded into S1 and passed to memory through the ALU. At T[4], RF OutASel is set according to RSEL and MuxBSel is set to 00 (ALUOut). ARF RegSel is set to 011 (PC) and ARF FunSel is set to 010 (load) so the value coming from Rx is loaded into PC.

BL operation takes only one cycle to execute. At T[3], ARF OutDSel is set to 11 (SP), Mem WR is set to 0 (read) and MuxBSel is set to 10 (MemOut). ARF RegSel is set to 011 (PC) and ARF FunSel is set to 010 (load), this way M[SP] is loaded into PC.

LDRIM also lasts one cycle. Again at T[3], MuxASel is set to 11 (IR[7:0]) and RF

RegSel is determined according to the RSEL value. RF FunSel is set to 100 so the address is written into the Rx register.

And lastly for STRIM operation, at T[3], the address value is loaded into S1 by setting MuxASel to 11, RF ScrSel to 0111 (S1) and RF FunSel to 010 (load). ALU FunSel is set to 10100 (A+B). At T[4], AR is loaded into S2 by setting ARF OutCSel to 10 (AR), MuxASel to 01 (OutC) and RF ScrSel to 1011 (S2). The contents of S1 and S2 are sent to ALU for addition by setting RF OutASel to 100 (S1) and RF OutBSel to 101 (S2). MuxBSel is set to 00 (ALUOut), ARF RegSel is set to 101 (AR) and ARF FunSel is set to 010 (load) so the addition is loaded back into AR. ARF OutDSel is also set to 10 (AR). At T[5], RF OutASel is determined according to RSEL and ALU FunSel is set to 10000 (A). MuxCSel is set to 0 and Mem WR is set to 1 (write) so the content of Rx is written into the memory.

2.3.2 Operations without Address Reference

Operations without Address Reference follow general format that was given in the Project file. Address is 16-bit long and contains the following parts:

OPCODE (6-bit)	S (1-bit)	DSTREG(3-bit)	SREG1 (3-bit)	SREG2 (3-bit)
----------------	-----------	---------------	---------------	---------------

Figure 2: Format of addresses without Address Reference

- **OPCODE** - 6-bit code that specifies which operation will be executed
- **Bit S** - which determines whether ALU Flags will be changed or not
- **DSTREG** - 3 bits specifying register which will store result of the operation
- **SREG1** - 3 bits specifying register source of the first operand in the operation
- **SREG2** - 3 bits specifying register source of the second operand in the operation

To begin with, before approaching to write any of the code that would execute any of the instructions, it was analyzed in detail how each of the instructions behave. This especially refers to the number of cycles every instruction requires to be executed, and therefore when the counter should be reset. In conclusion, it was decided that instructions without address reference (whose total number is 21) will be divided into 2 groups (one of them containing one additional subgroup). Those groups are as following:

- **instructions with one operand**
- **instructions with two operands**

When executing these types of instructions it is sufficient to consider each of the 3 registers whose code is given through groups of 3 bits in the instruction. Here, firstly, source registers were considered. In this segment, it is important the detail that we have two different types of instructions, because for those 1-operand ones it is enough to examine SREG1 part. However, for the ones with two operands SREG2 is taken in consideration, as well. On the picture provided below, it is given the first part of the code that ensures that register codes are separated from the instruction. Also, "sources" part is crucial when deciding whether the particular register is the part of RegisterFile (RF) or AddressRegisterFile (ARF). Based on this there are 4 possible cases, so either both operands are from the same place (RF or ARF) or from different.

```

wire Z = ALU_FlagOut[3];
wire [2:0] DSTREG = {RSEL[0], IROut[7:6]};
wire [2:0] SREG1 = IROut[5:3];
wire [2:0] SREG2 = IROut[2:0];

// Sources:
// 11 : RF, RF
// 10 : RF, ARF
// 01 : ARF, RF
// 00 : ARF, ARF
reg [1:0] sources = 2'b11;

```

Figure 3: DSTREG, SREG1, SREG2 decoding

Also it is important to mention that instructions D[24], D[25], D[26], D[27], D[28], D[29] require that ALUFlag's WF input is enabled so that the flags can be changed.

Next, SREG1 is considered. Based on the ALU Unit constructed in the Project 1, it is obvious that when source register is one of the RFs, its content can be directly transmitted to the ALU via OutA for SREG1 and OutB for SREG2. That case is simpler one and requires only one clock cycle. However, when source register is in ARF then there is no direct connection to ALU and the sending is done using Scratch registers of RF and MUXA. For this case 2 clock cycles are needed. The most special case is if the both of the operands are taken from ARF. Then this needs to be done in 3 clock cycles. It should be remarked once again that SREG2 should not be considered for one-operand instructions: D[5], D[6], D[7], D[8], D[9], D[10], D[11], D[14], D[24].

Additionally, it is required to analyze the DSREG. Again the two cases are encountered, when the register is in RF and when it is in ARF. In either case the one clock cycle is needed to send data from ALU output to the particular destination register. The only thing that makes difference is that data is sent to RF through MUXA while to ARF through MUXB.

The last special cases, or the mentioned subgroup, are instructions for increment and decrement, D[5] and D[6], respectively.

```

// Incrementing and decrementing operations
// In case of increment, increment DSTREG, it's already swapped with SREG1
if (((T[4] && sources == 2'b11) ||
    ((T[5] && sources == 2'b01) && D[5]) // (T[5] && sources == 2'b10) ||
    // (T[6] && sources == 2'b00)
begin
    if (DSTREG[2] == 0) begin // If DSTREG is in ARF
        ARF_FunSel <= 3'b001; // Increment
    end else begin // If DSTREG is in RF
        RF_FunSel <= 3'b001; // Increment
    end
end
// In case of decrement, decrement DSTREG, it's already swapped with SREG1
if (((T[4] && sources == 2'b11) ||
    (T[5] && sources == 2'b01) && D[6]) // (T[5] && sources == 2'b10) ||
    // (T[6] && sources == 2'b00) &&
begin
    if (DSTREG[2] == 0) begin // If DSTREG is in ARF
        ARF_FunSel <= 3'b000; // Decrement
    end else begin // If DSTREG is in RF
        RF_FunSel <= 3'b000; // Decrement
    end
end
end

```

Figure 4: Increment and Decrement instructions code

As can be seen from the picture provided below, the INC and DEC instructions are handled in a way that the first part of selecting SREG1 and DSREG is common with all other 1-operand instructions while the only special thing is that after sending the value to destination register it is being incremented or decremented using FunSel code.

2.4 Resetting

Reset is the essential part of the execution of every instruction. It is important to ensure that after each instruction cycle counter is reset to its initial value so that next instruction coming will have its own cycle started from T1 (that way enabling fetching and decoding). The code used for this part is provided below:

```

if ((T[5] && D[30] || D[31] || D[43]) || (T[6] && D[0] || D[1] && D[2] || D[12] && D[3]) ||
    (T[5] && D[7] || D[8] || D[9] || D[30] || D[31] || D[37] || D[18] || D[19] || D[20] || D[31] || D[32]) ||
    (T[5] && D[33]) ||
    ((D[5] || D[6] || D[7] || D[8] || D[9] || D[30] || D[31] || D[34] || D[24]) && ((T[5] && sources[1] || (T[6] && sources[1])) ||
    (D[12] || D[13] || D[15] || D[16] || D[17] || D[23] || D[25] || D[26] || D[27] || D[28] || D[29] &&
    (T[4] && sources[1] && sources[0]) || (T[5] && (sources[1] ~ sources[0])) || (T[6] && (sources[1] && sources[0]))))
begin
    SC_Reset <= 1; // Reset the counter to zero
    RF_RegSel <= 4'bZZZZ;
    RF_AccSel <= 4'bZZZZ;
    ARF_RegSel <= 3'bZZZ;
    RF_RegSel <= 3'bZZZ;
    RF_FunSel <= 3'bZZZ;
    Mem_CS <= 1;
    Mem_Alt <= 0;
    MuxA_Sel <= 2'bZZ;
    MuxB_Sel <= 2'bZZ;
    MuxC_Sel <= 1'bZ;
end
end

```

Figure 5: Reset code

Here, based on the clock cycles and the duration of execution of each instruction, previously explained in details, reset conditions have been written. The crucial thing at this point is that SC Reset is set to 1, so reset is done.

3 RESULTS

For the testing purposes, we have used the binary code provided in the Project file. Based on the given instructions, following the formats given for the instructions, RAM.mem file was prepared. It is further used when testing the results of our Control Unit System Code. Below are provided some example outputs obtained when evaluating out code.


```

Output Values:
T: 1
Address Register File: PC: 0, AR: x, SP: x
Instruction Register : x
Register File Registers: R1: 0, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: x, N: x, C: x, O: x
ALU Result: ALUOut: 0

Output Values:
T: 2
Address Register File: PC: 1, AR: x, SP: x
Instruction Register : X
Register File Registers: R1: 0, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: x, N: x, C: x, O: x
ALU Result: ALUOut: 0

Output Values:
T: 4
Address Register File: PC: 2, AR: x, SP: x
Instruction Register : 40
Register File Registers: R1: 0, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: x, N: x, C: x, O: x
ALU Result: ALUOut: x

```

Figure 6:

```

Output Values:
T: 8
Address Register File: PC: 2, AR: x, SP: x
Instruction Register : 40
Register File Registers: R1: 0, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 40, S2: 0, S3: 0, S4: 0
ALU Flags: Z: x, N: x, C: x, O: x
ALU Result: ALUOut: x

Output Values:
T: 16
Address Register File: PC: 2, AR: x, SP: x
Instruction Register : 40
Register File Registers: R1: 0, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 40, S2: 2, S3: 0, S4: 0
ALU Flags: Z: x, N: x, C: x, O: x
ALU Result: ALUOut: 42

Output Values:
T: 32
Address Register File: PC: 42, AR: x, SP: x
Instruction Register : 40
Register File Registers: R1: 0, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 40, S2: 2, S3: 0, S4: 0
ALU Flags: Z: x, N: x, C: x, O: x
ALU Result: ALUOut: 42

```

Figure 7:

4 DISCUSSION

During this project, we tried to design a CPUSystem module that fetches, decodes and executes instructions from a memory, similar to how a real CPU operates but in a much simpler way. We had already designed certain modules like registers and register files in the previous project and it was helpful for understanding the structure of a computer. But we can say that during this second project we had the chance to understand how these parts of the computer interact with each other to perform certain operations and store data. But aside from giving us a better understanding of a computer, this project was quite a challenge for us considering how hard it was trying to implement a working system.

5 CONCLUSION

To sum up everything done throughout this project it is possible to say that it was highly challenging. This comes from the fact that there was a large number of instructions to be implemented and although some of them have some common properties, each one of these needed to be studied separately. In that way it was highly time-consuming. Furthermore, the part we have struggled the most with was understanding the way in which clock acts. Everything in the project is so tightly related to each other, and when one part is not working correctly the other part are directly being affected, as well. However, it was great way to use all the theoretical facts we were seeing in classes and try to implement our own control unit model.

REFERENCES